

(Vanilla) Recurrent Neural Networks

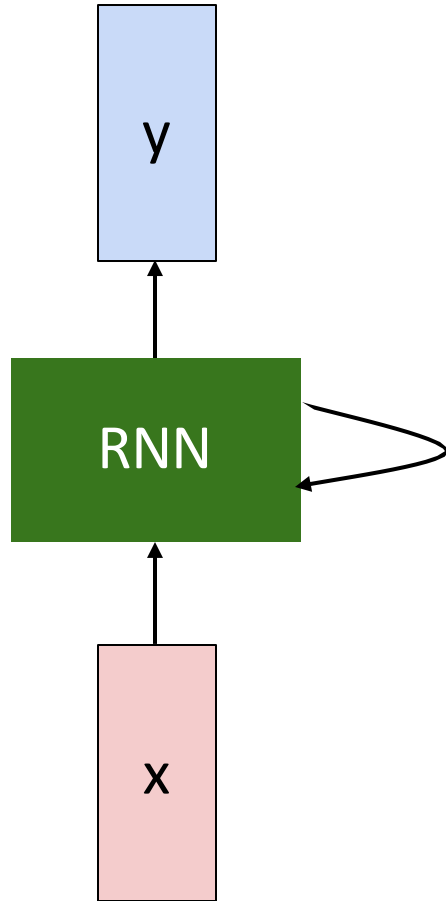
The state consists of a single “*hidden*” vector $\mathbf{h}$:

$$h_t = f_W(h_{t-1}, x_t)$$

(also bias term)

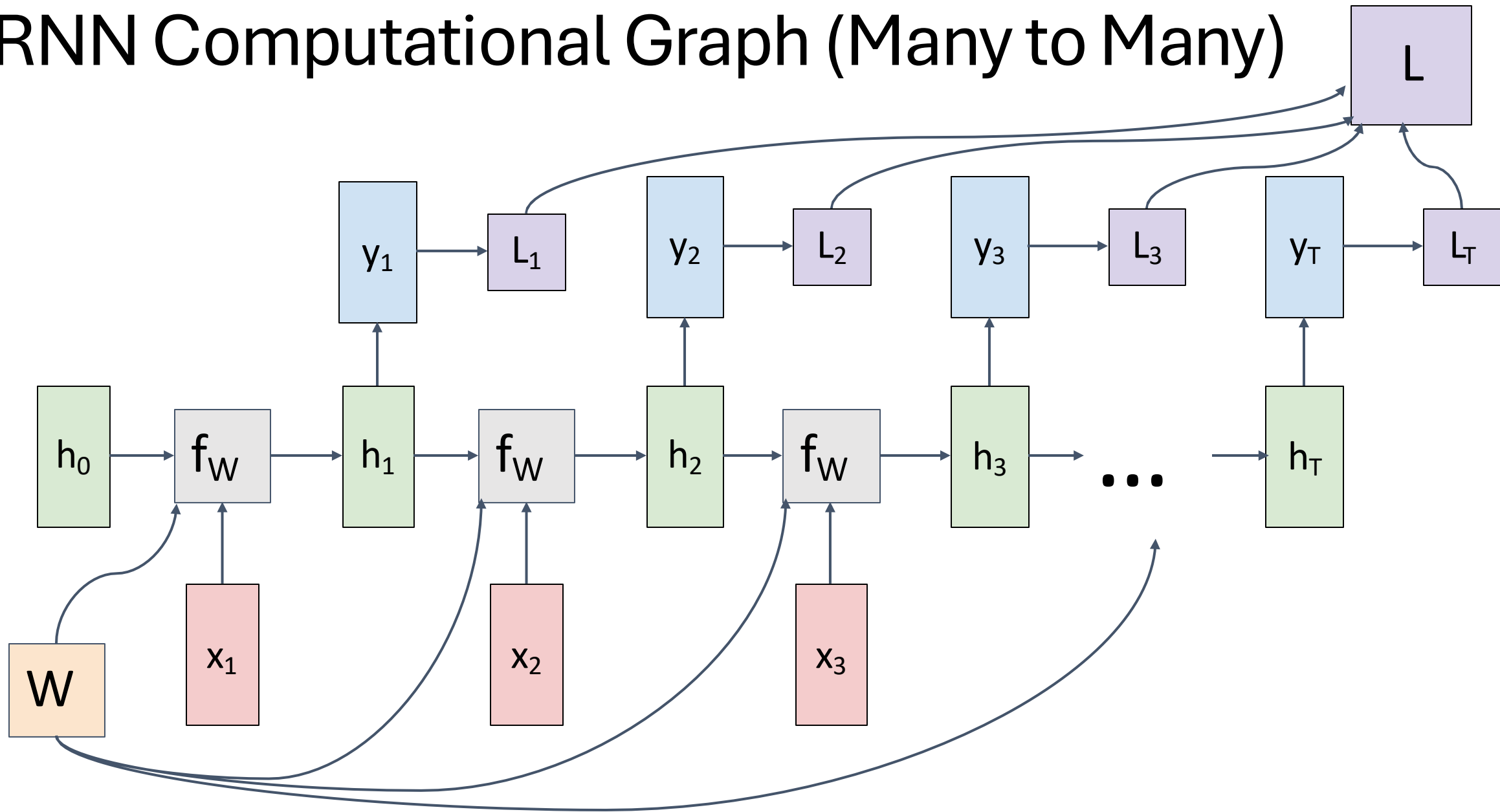
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t$$



Sometimes called a “Vanilla RNN” or an “Elman RNN” after Prof. Jeffrey Elman

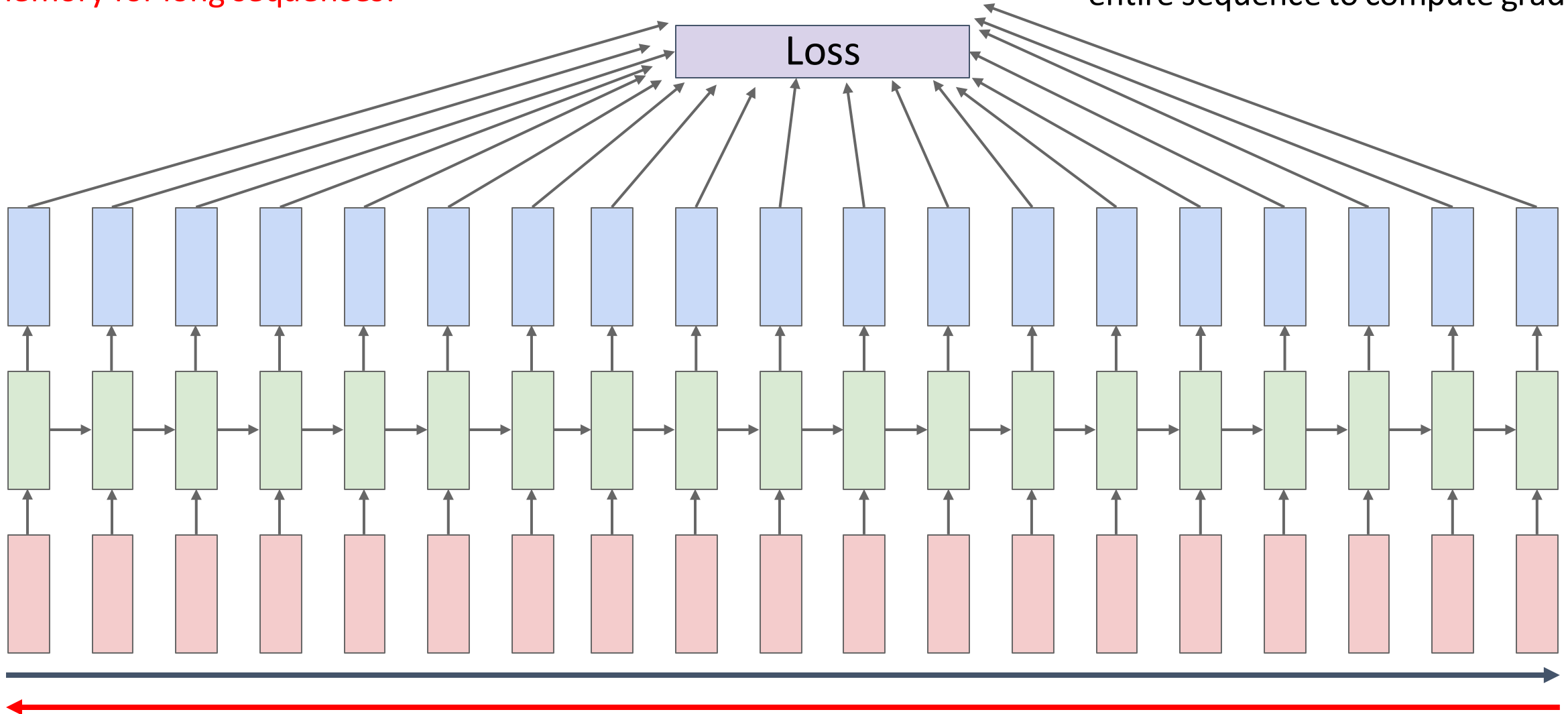
RNN Computational Graph (Many to Many)



Appendix: Backpropagation Through Time

Problem: Takes a lot of
memory for long sequences!

Forward through entire sequence to
compute loss, then backward through
entire sequence to compute gradient

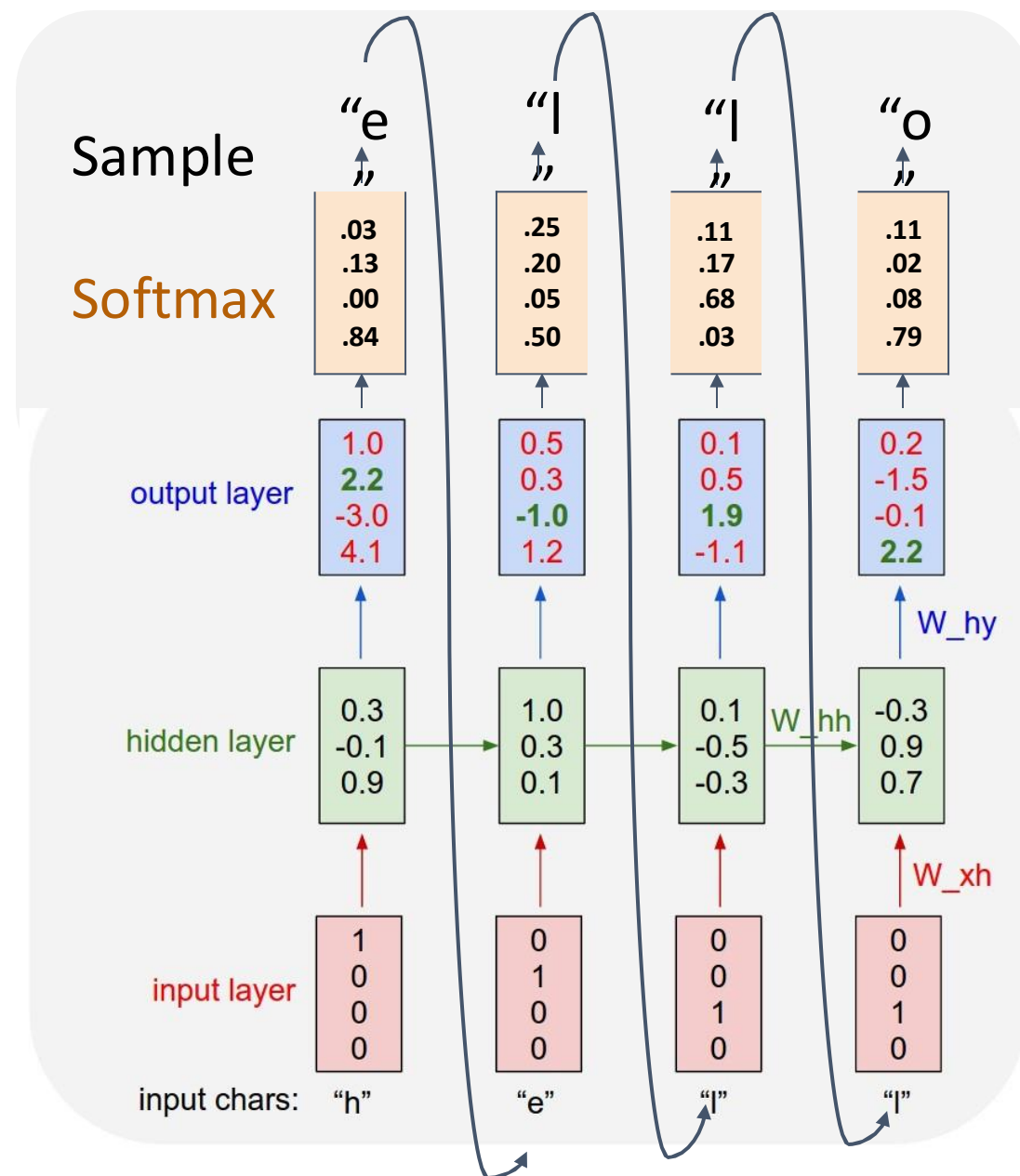


Example: Language Modeling

At test-time, **generate** new text: sample characters one at a time, feed back to model

Training sequence: "hello"

Vocabulary: [h, e, l, o]



at first:

tyntd-iafhatawiaoihrdemot lytdws e ,tfti, astai f ogoh eoase rrranbyne 'nhthnee e
plia tklrqd t o idoe ns,smtt h ne etie h,hregtrs nigtike,aoaenns lng

↓ train more

"Tmont thithey" fomesscerliund
Keushey. Thom here
sheulke, anmerenith ol sivh I lalterthend Bleipile shuw y fil on aseterlome
coaniogennc Phe lism thond hon at. MeiDimorotion in ther thize."

↓ train more

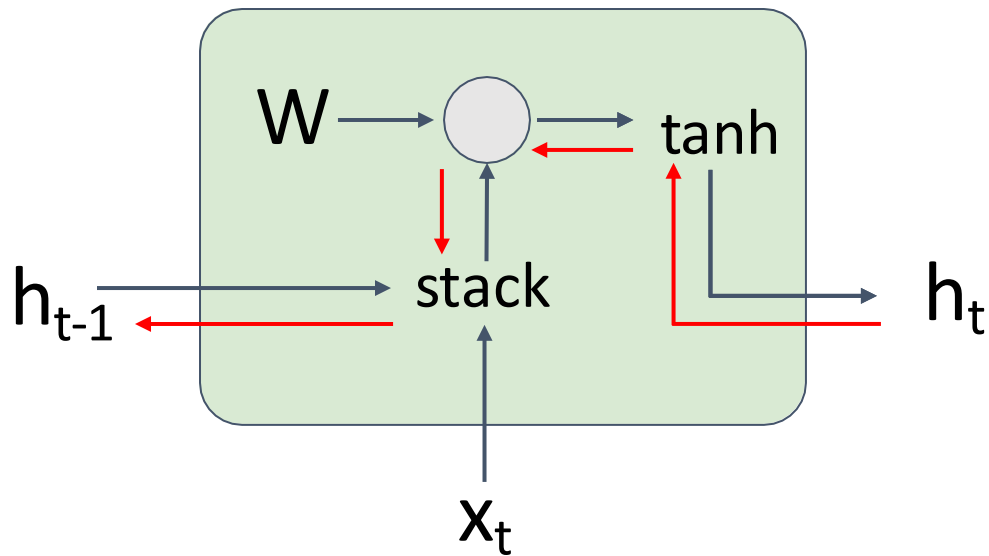
Aftair fall unsuch that the hall for Prince Velzonski's that me of
her hearly, and behs to so arwage fiving were to it beloge, pavu say falling misfort
how, and Gogition is so overelical and ofter.

↓ train more

"Why do what that day," replied Natasha, and wishing to himself the fact the
princess, Princess Mary was easier, fed in had oftened him.
Pierre aking his soul came to the packs and drove up his father-in-law women.

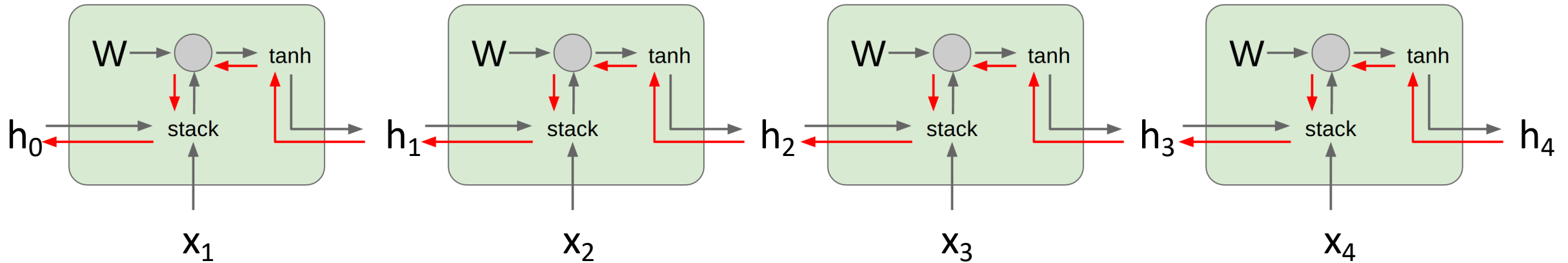
Vanilla RNN Gradient Flow

Backpropagation from h_t to h_{t-1} multiplies by W (actually W_{hh}^T)



$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{hx}x_t) \\ &= \tanh\left((W_{hh} \quad W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\ &= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \end{aligned}$$

Vanilla RNN Gradient Flow



Computing gradient of h_0 involves many factors of W (and repeated $\tanh$)

Largest singular value > 1 :
Exploding gradients

Largest singular value < 1 :
Vanishing gradients

→ **Change RNN architecture!**

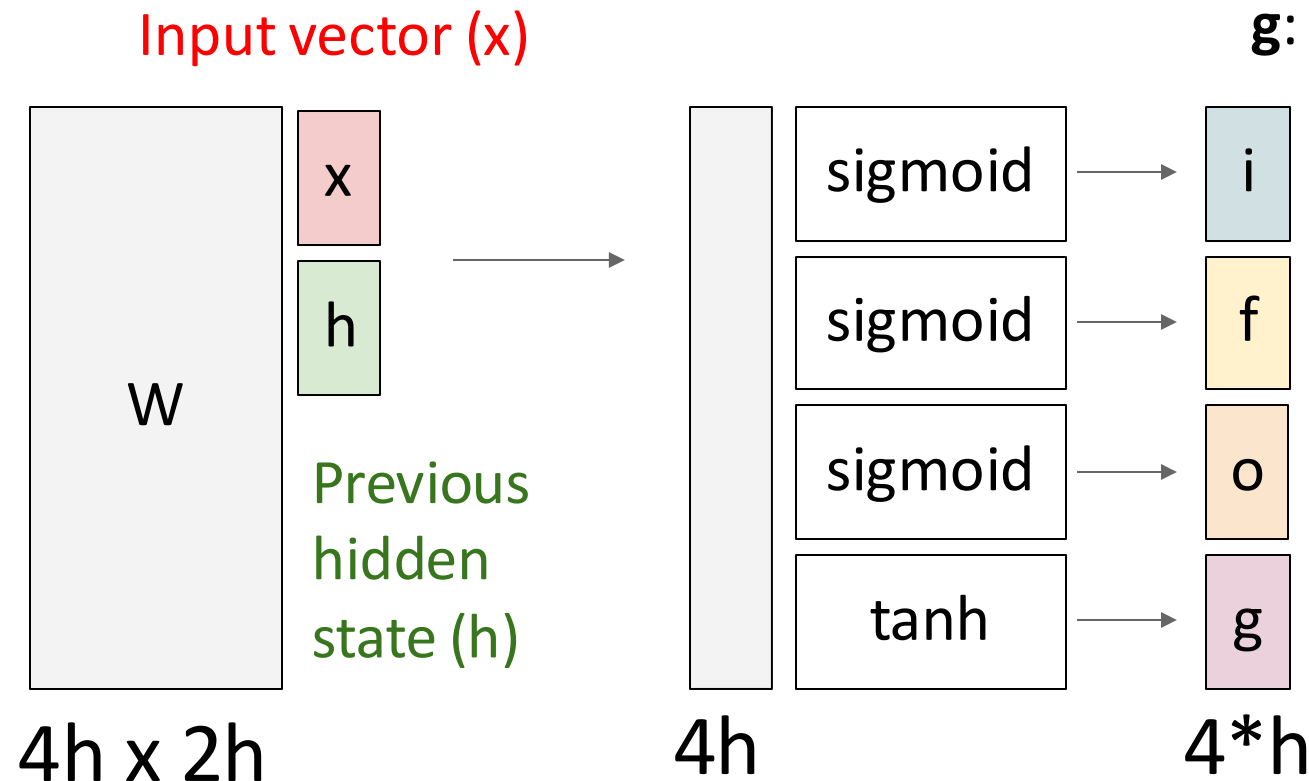
Long Short Term Memory (LSTM)

i: Input gate, whether to write to cell

f: Forget gate, Whether to erase cell

o: Output gate, How much to reveal cell

g: Gate gate (?), How much to write to cell

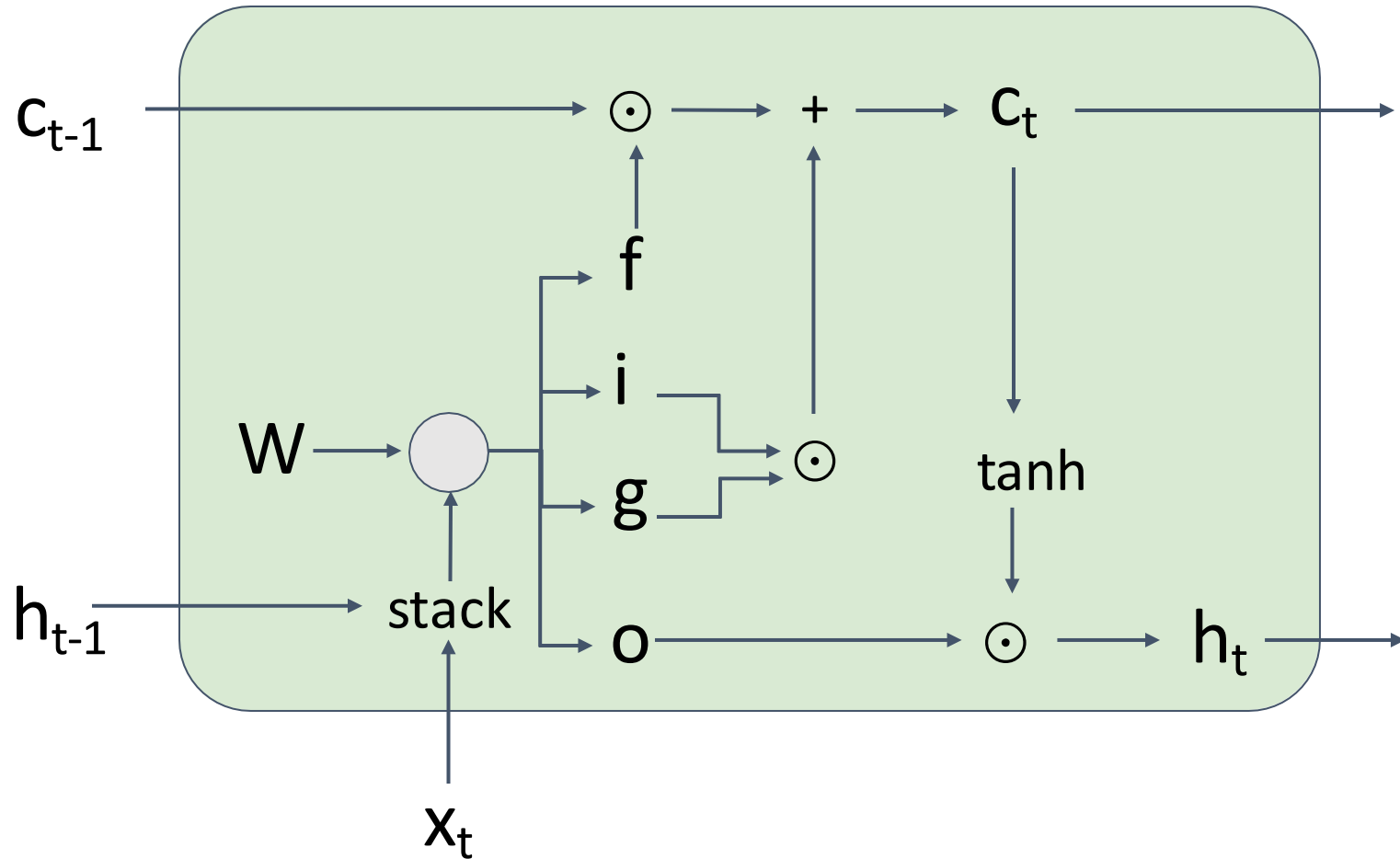


$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM)

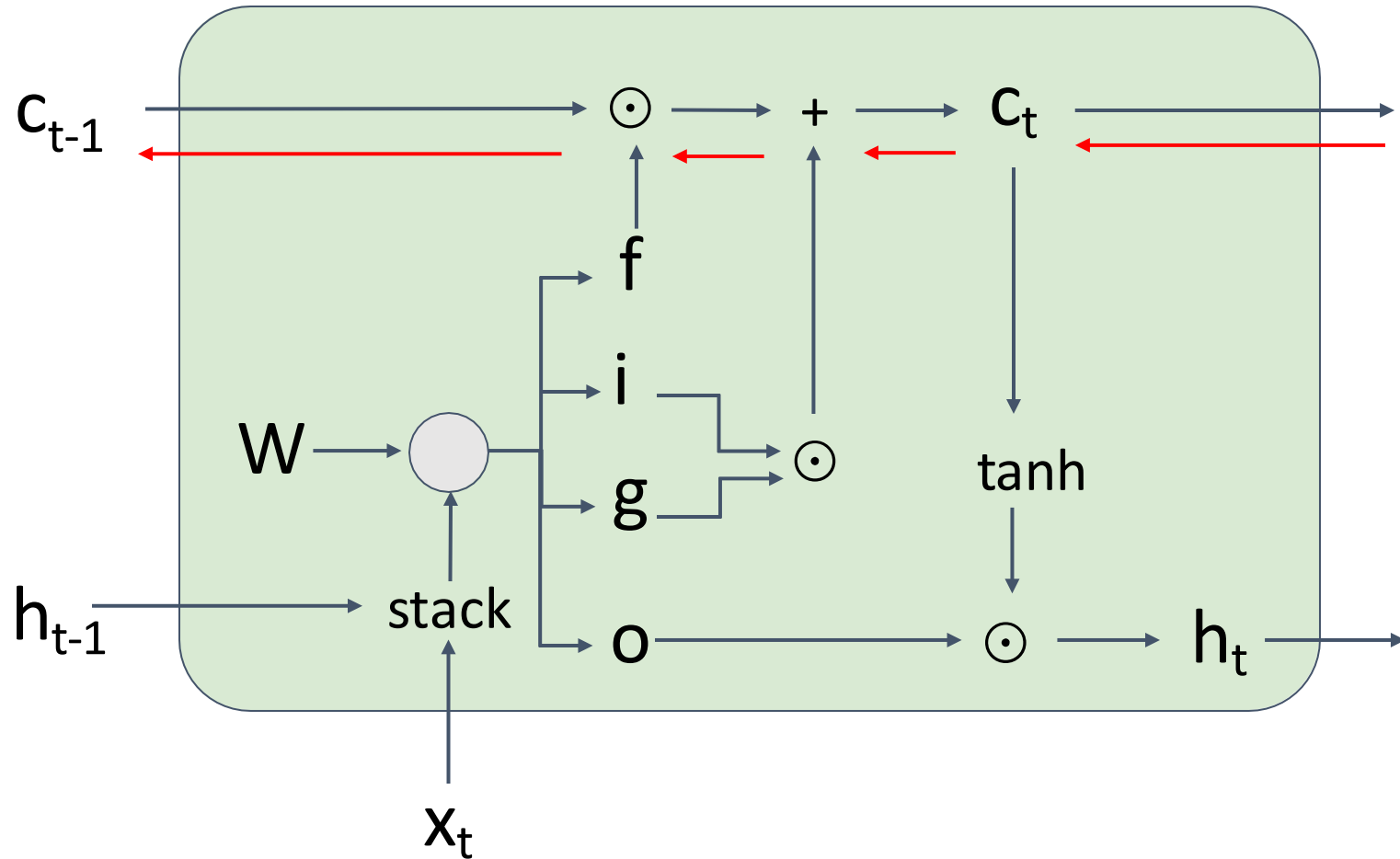


$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM): Gradient Flow



Backpropagation from c_t to c_{t-1} only elementwise multiplication by f , no matrix multiply by W

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

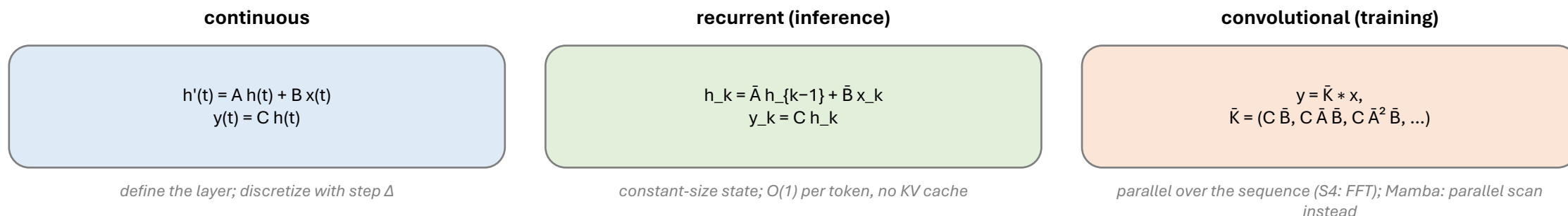
$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

H3: Hungry Hungry Hippos — state-space layers as a drop-in for attention

- Before H3, SSM language models (S4) trailed Transformers by several perplexity points; H3 diagnosed two missing skills: **recalling** earlier tokens and **comparing** tokens across the sequence
- H3 stacks two SSMs with multiplicative gating (a shift SSM and a diagonal SSM) so the layer can implement recall and comparison — the same skills attention's induction heads provide
- With just two attention layers added, a hybrid H3 model matched or beat Transformers on OpenWebText perplexity (125M–2.7B); FlashConv (FFT-based) made the SSM convolution fast
- This line led to Mamba (selective SSMs, 2023), Mamba-2 (SSD, 2024) and Gated DeltaNet (2025) — the linear layers used in today's hybrids (next slides)

State-space layers: three views of one model



Mamba (2023): make B , C , Δ functions of the input x_k (selective) — the recurrence can now forget or keep per token, at the cost of the convolution view

- Continuous: $h' = A h + B x$; discretize with a step Δ
- Recurrent (inference): a fixed-size state, $O(1)$ per token, no KV cache
- Convolutional (training): the same map as a long convolution — S4 computes it with FFTs in parallel
- Mamba (Gu & Dao, 2023): B , C , Δ become functions of the input (selective) — the layer can forget or keep per token; training uses a parallel scan instead of the convolution

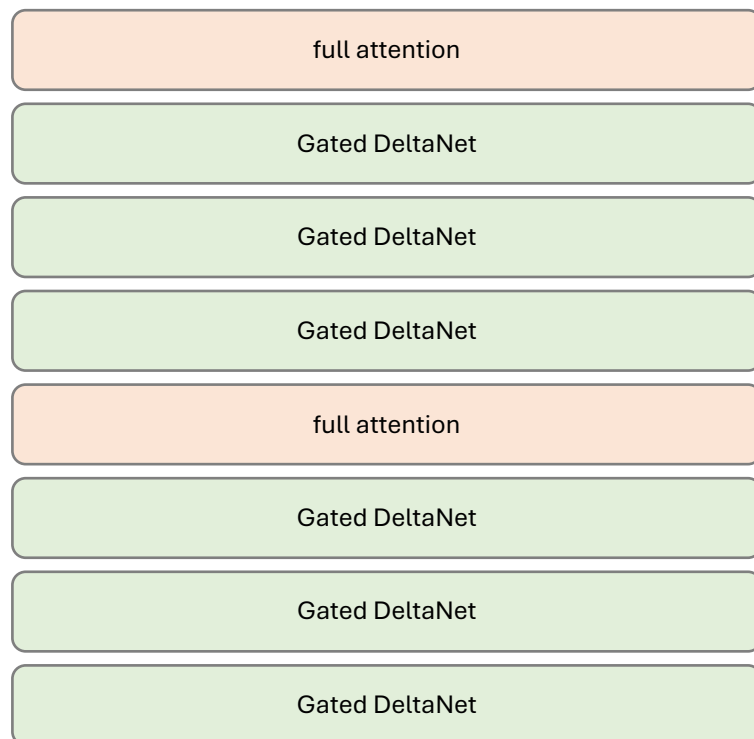
Why SSMs, and why not only SSMs

	training (parallel over T)	inference (per new token)	state / memory	exact recall
recurrent (RNN)	slow (sequential)	fast, $O(1)$	fixed	weak
attention	fast	slow, $O(T)$ + KV cache	grows with T	strong
SSM / linear attention	fast (conv / scan)	fast, $O(1)$	fixed ($d \times d$ per head)	weak $\rightarrow$ hybrids

- Mamba (2023) was the first attention-free LM to match Transformer scaling on language, with linear-time training and constant-memory generation
- Linear attention (drop the softmax) is the same object: a recurrent state $S_t = S_{t-1} + v_t k_t^T$ — Mamba-2's SSD makes the connection exact (Dao & Gu, 2024)
- Weakness: a fixed-size state cannot store every past token — exact recall (needle-in-a-haystack, copying) is worse than attention
- Hence hybrids: keep a few attention layers for recall, use SSM / linear layers for the rest (Jamba 2024 $\rightarrow$ Nemotron-H 2025 $\rightarrow$ Qwen3-Next, Kimi Linear, Nemotron 3)

Hybrids: attention + linear recurrence

3 : 1 pattern (Qwen3-Next / Qwen3.5)



linear: fixed state, $O(T)$; attention: exact recall, KV cache

- Linear-attention / state-space layers (Mamba-2, Gated DeltaNet) keep a constant-size state: no KV growth, $O(T)$ time — but weaker at exact recall, so a few full-attention layers stay
- **Qwen3-Next** (Sep 2025) and **Qwen3.5** (Feb 2026): 3 Gated DeltaNet : 1 gated attention
- **Kimi Linear** (Oct 2025): 3 Kimi Delta Attention : 1 MLA, 1M context; **Nemotron 3** (Dec 2025 – Mar 2026): mostly Mamba-2 with a few GQA layers, 1M context
- **Ling 2.5** (Feb 2026): 7 Lightning Attention : 1 MLA; **xLSTM-7B** (2025): no attention at all
- Gallery trend: dense $\rightarrow$ sparse MoE $\rightarrow$ hybrid (most hybrids are also MoE)

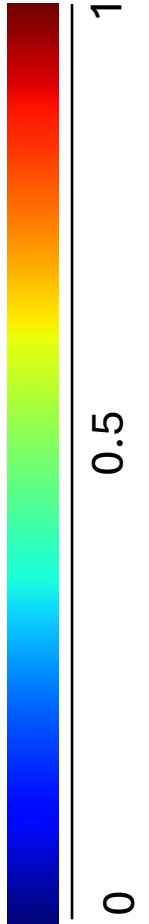
Co-Attention for Visual Question Answering



What **color** is the plastic **container**
inside the cage?



What **animal** is **in the** cage?



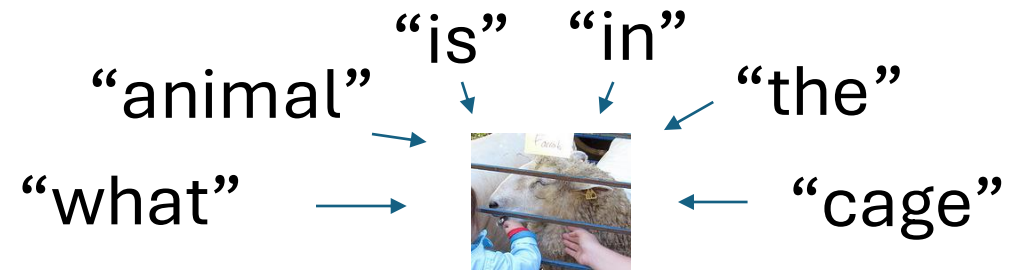
Factorize the attention belief

$$b_I(I, Q, i) \propto \exp(w_I \psi_I(i) + w_{I,Q} \mu_{Q \rightarrow I}(i))$$

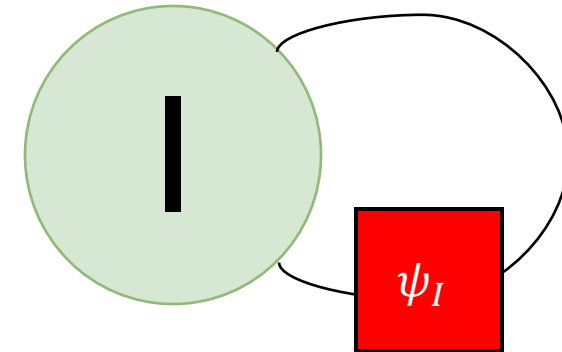
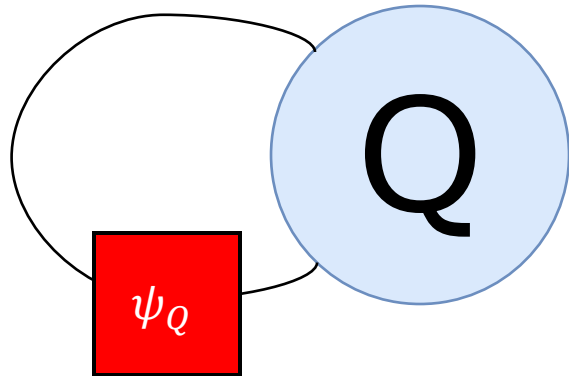
Unary ($\psi_I(i)$): region information



question-to-Image ($\mu_{Q \rightarrow I}(i)$):
interactions of regions with words



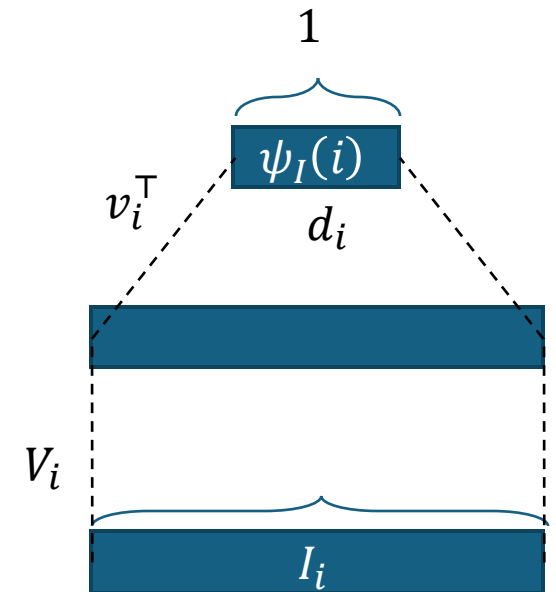
Unary factor



For each image region:

$$\psi_I(i) = v_i^\top \text{relu}(V_i I_i)$$

Where weights $v_i^\top \in \mathbb{R}^{d_I \times 1}$ and $V_i \in \mathbb{R}^{d_I \times d_I}$



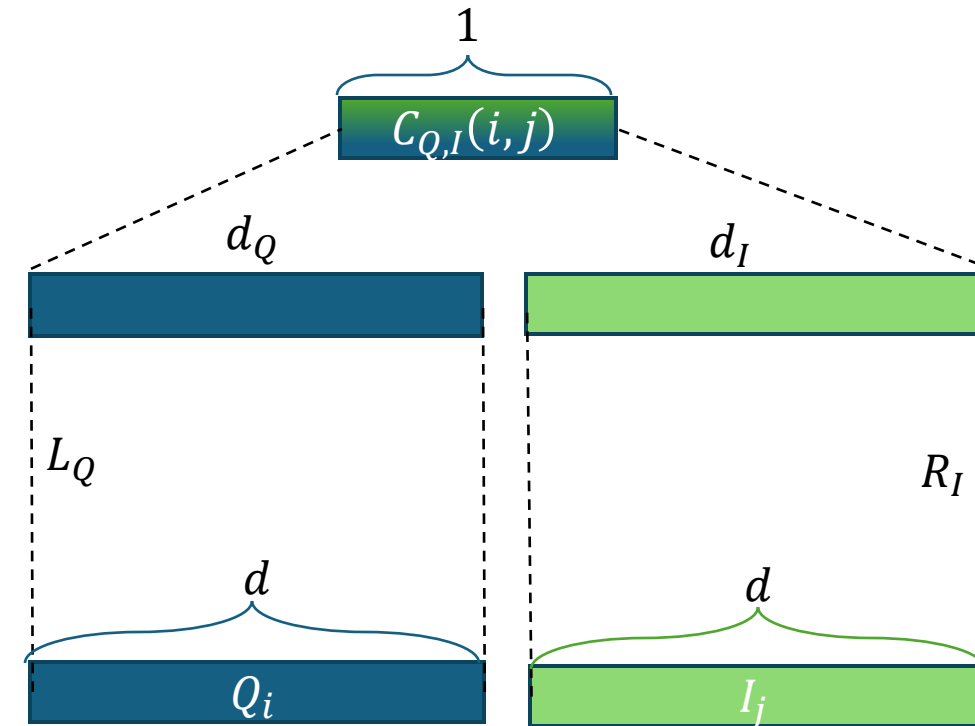
$$b_I(I, Q, i) \propto \exp(w_I \psi_I(i) + w_{I,Q} \mu_{Q \rightarrow I}(i))$$

The interaction is learned

- For each two modalities

$$C_{Q,I}(i,j) = \left(\frac{L_Q Q_i}{\|L_Q Q_i\|} \cdot \frac{R_I I_j}{\|R_I I_j\|} \right)$$

- Where weights $L_q \in \mathbb{R}^{d_Q \times d}$ and $R_i \in \mathbb{R}^{d_I \times d}$



$$b_I(I, Q, i) \propto \exp(w_I \psi_I(i) + w_{I,Q} \mu_{Q \rightarrow I}(i))$$

Message from question to image

- Messages via weighted marginalization: $\mu_{Q \rightarrow I}(i) = \sum_{Q_j} W_{I,i,j} C_{Q,I}(i,j)$

	$C_{Q,I}(i,j)$				$\mu_{Q \rightarrow I}(i)$	
						2.1
What	1.2	2.3	1.7	$\sum^w$ 		9
animal	-0.4	6.5	4.2			6.5
is	1.3	1.2	0.6			

$$b_I(I, Q, i) \propto \exp(w_I \psi_I(i) + w_{I,Q} \mu_{Q \rightarrow I}(i))$$